

1. A script is code you keep

Create one with **File** → **New File** → **R Script**. Type:

A script is the right place for a function you will call from more than one Quarto document. Copy-paste means every fix has to be repeated.

2. Write the function once

```

#' Geometric mean of a positive numeric vector
#'
#' Uses log(), mean(), and exp().
#'
#' @param x A vector of positive numbers.
#'
#' @return A single numeric value.
#'
#' @examples
#' geo_mean(c(1, 6, 2, 5))
geo_mean <- function(x) {
  stopifnot(is.numeric(x))
  x |>
    log() |>
    mean() |>
    exp()
}
```

If you prefer `%>%`, load `magrittr` or `tidyverse` in the script (or in the file that sources it) before you call `geo_mean()`.

The three classroom functions from note **2.2** are the same kind of file: one function, Roxygen header, then the body. Download them and save them as `R/add_half.R`, `R/fib.R`, and `R/rep_even.R`:

- [add_half.R](#)
 - [fib.R](#)
 - [rep_even.R](#)
-

3. `source()` makes the function available

`source()` runs a script in the current session. After that, the functions it defined are objects you can call.

Use a **relative** path from the project, the same rule as note 1.3:

```
source("R/geo_mean.R")

geo_mean(c(1, 6, 2, 5))

source("R/fib.R")
fib(7)
# [1] 13
```

From a Quarto file in `analysis/`, the relative path might be `../R/geo_mean.R`. `here::here("R", "geo_mean.R")` is another reliable option once you are in an RStudio project.

If the script uses `%>%` or `ggplot()`, those packages must be loaded in the session. Either put `library(...)` at the top of the script, or load the packages in the document before `source()`. Packages are the durable way to manage those dependencies. DATA 413/613 is the course that builds them.

4. Practice

Write a function that takes a numeric vector and returns `TRUE` where the value is between 0.1 and 0.5, inclusive of the endpoints if you like, and `FALSE` otherwise.

1. Save it in `R/` under a name that matches the function.
 2. In a Quarto document (or an R script) in another folder, `source()` that file with a relative path.
 3. Call it on `c(0, 0.2, 0.4, 0.7)`.
-

5. This week, in one place

By the end of Week 2 you should be able to:

- use `|>` or `%>%` for a sequence of steps
- write a function: small example first, name it, give arguments and defaults, return a value, check inputs, document it
- branch with `if / else / else if`, or `switch()` when one argument picks among named options
- save the function in an `.R` script and `source()` it from another file

Functions and helpers from this week include `function()`, `set.seed()`, `rnorm()`, `sort()`, `diff()`, `round()`, `mean()`, `range()`, `stopifnot()`, `any()`, `all()`, `&&`, `||`, `switch()`, `source()`, `fib()`, `rep_even()`, and `add_half()`. For floating-point equality, `dplyr::near()` is safer than `==`.